



Transações e Controle de Concorrência

Banco de Dados

Prof. Dr. Carlos Melo

 carlos.melo@upe.br



O Problema

- Como o banco garante consistência e segurança durante operações simultâneas?
 - Transações
 - COMMIT e ROLLBACK
 - ACID
 - Concorrência
 - Isolamento
- Sistemas reais executam de milhares a milhões de operações simultâneas:
 - bancos, e-commerce, reservas, pagamentos, etc.



O Problema

- Concorrência

produto	estoque
PS5	1

- Duas pessoas compram o produto ao mesmo tempo.
- O que pode acontecer?
 - Usuários A/B veem na aplicação a mensagem APENAS 1 DISPONÍVEL
 - Ambos conseguem clicar em comprar (mas há apenas um item)
- O banco precisa controlar ambas as operações



O Problema

- O controle se dá por meio de Transações, que nada mais são do que um conjunto de operações executadas como uma única unidade lógica
- Transferência bancária:
 - 1. remove dinheiro da conta A
 - 2. adiciona dinheiro na conta B
- E se o sistema falhar no meio? Dinheiro saiu de A e não chegou a B...
 - Sumiu?



BEGIN e COMMIT e ROLLBACK

- **BEGIN** inicia uma transação que é confirmada definitivamente por um **COMMIT**

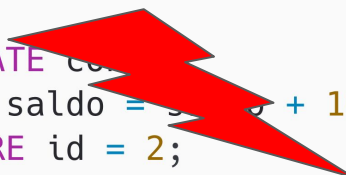
```
1 BEGIN;  
2  
3 UPDATE conta  
4 SET saldo = saldo - 100  
5 WHERE id = 1;  
6  
7 UPDATE conta  
8 SET saldo = saldo + 100  
9 WHERE id = 2;  
10  
11 COMMIT;
```



BEGIN e COMMIT e ROLLBACK

- Já o **ROLLBACK** desfaz as alterações de uma transação incompleta

```
1 BEGIN;  
2  
3 UPDATE conta  
4 SET saldo = saldo - 100  
5 WHERE id = 1;  
6  
7 UPDATE conta  
8 SET saldo = saldo + 100  
9 WHERE id = 2;  
10  
11 COMMIT;
```





O Problema

- Não existe meio do caminho em operações **ACID**
 - Ou a transação ocorre completamente ou não ocorre (**Atomicidade**).
- **Consistência**
 - A transação deve manter o banco em um estado válido
 - Salário não deve ser menor que o mínimo,
 - Não pode haver chave estrangeira que aponta para registro inexistente, etc.
- Isso já vem da própria modelagem do banco e é garantido pelo SGBD



O Problema

- **Isolamento**

- Transações simultâneas não devem interferir uma na outra
- O banco tenta garantir isso
- Dois não deverão conseguir comprar o mesmo produto ao mesmo tempo
- O próprio banco controla concorrência para reduzir conflitos simultâneos

- **Durabilidade**

- Após o **COMMIT** os dados devem permanecer salvos
- Para uma nova alteração só a partir de uma nova transação



Problemas de Concorrência

- Quando múltiplas transações:
 - leem
 - alteram
 - removem
- Dados simultaneamente, problemas podem acontecer

Problema	Descrição
Dirt Read	Leitura de dados não confirmados
Lost Update	Atualização perdida
Non-repeatable Read	Mesma consulta retorna valores diferentes



Dirty Read

- Uma transação lê dados que ainda não foram *committados*

```
1 UPDATE conta  
2 SET saldo = 0  
3 WHERE id = 1;
```

- Mas a transação de atualização ainda não executou um **COMMIT**
- Então uma transação B pode ler saldo = 0
- Caso a transação A execute um ROLLBACK então o saldo 0 nunca foi committado e a transação B fez uma leitura suja



Lost Update

- Uma atualização sobrescreve outra sem perceber
- As transações A e B lêem 1000

```
1 A lê 1000
2 B lê 1000
3
4 A salva 900
5 B salva 800
6
7 900 DESAPARECE
```



Non-Repeatable Read

- A mesma transação executa a mesma consulta duas vezes, mas outra transação alterou os dados no meio
- A transação A lê a primeira vez e tem 1000 como saída:

```
1 SELECT saldo
2 FROM conta
3 WHERE id = 1;
```

- A transação B atualiza saldo para 500
- A transação B lê pela segunda vez e a saída será 500



Níveis de Isolamento

- OS SGBDs oferecem diferentes níveis de isolamento
- O objetivo é:
 - Equilibrar concorrência com desempenho e segurança

Nível	Proteção
Read Uncommitted	Mínima
Read Committed	Evita dirty read
Repeatable Read	Evita releitura inconsistente
Serializable	Maior isolamento



Read Uncommitted

- Menor nível de isolamento
- Permite leituras de dados ainda não confirmados
- Dirty Reads podem ocorrer
- Permite uma maior concorrência e menor bloqueio
 - Há um risco maior de inconsistência



Read Committed

- Uma transação só pode ler dados confirmados
- Evita Dirty Reads
- Porém, Non-repeatable Reads e Mudanças durante a transação ainda podem ocorrer
 - É o nível padrão em muitos SGBDs (Incluindo o PostgreSQL)



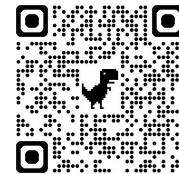
Repeatable Read

- Uma transação deve enxergar os mesmos dados durante toda a sua execução
- Evita Non-repeatable Read
 - Se a transação ver um saldo = 1000, ela continuará enxergando o saldo = 1000 mesmo que outra operação já tenha alterado o saldo para 900
 - Maior segurança e maior consistência
 - Porém, menor concorrência



Serializable

- Transações são executadas como se fossem sequenciais
- Mesmo com múltiplos usuários e operações o resultado final é equivalente a uma execução em série
 - Maior segurança e maior consistência
 - Porém, menor concorrência com um maior custo



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br